Automated Theorem Prover

Ariana Davey and Katelyn Toto

Dr. Hieu Bui

Department of Electrical and Computer Science, The Catholic University of America

CSC 210: Discrete Mathematics

May 5, 2025

1 Introduction

The Automated Theorem Prover that was developed for this project is called ValidityProver and its main focus is to evaluate the validity of propositional logic statements. Propositional logic statements are expressions that include statement variables and logical connectives. The variables *p, q, and r* are three examples of statement variables. Some examples of logical connectives include ~ *(not),* $\wedge$ *(and),* $\vee$ *(or),* $\rightarrow$ *(therefore),* and $\leftrightarrow$ *(if and only if).* The Automated Theorem Prover (ATP), ValidityProver, was developed using the programming language Python. Propositional logic is the building block for developing an understanding of the theory of computing. ValidityProver would be helpful for computer science students due to its simplicity and functionality. ValidityProver is able to parse logical arguments, generate truth tables, verify a statement's validity in real-world scenarios, and build trees to represent logical expressions.

ValidityProver has four main surface-level components. First, it parses propositional logic statements, using variables such as *A, B, C,* and *D* to represent logical propositions. In simpler terms, ValidityProver can break an expression up into these statement variables for easier evaluation. Secondly, ValidityProver generates a truth table for the given expression. Thirdly, ValidityProver uses the truth table values in order to analyze an expression's validity. Finally, the program displays a tree for each expression for easier comprehension. This research highlights the crucial role of automated theorem provers (ATPs) in the development of mathematical understanding and propositional logic.

2 Literature Review

Automated theorem provers prove to be significant through the available research on the topic. Currently, very few automated theorem provers (ATPs) exist that are accessible or user-friendly for students studying discrete mathematics. A recent tool for a guided practice of proposition proofs, called the LogicLearner, was developed in March 2025. Inamdar et al. (2025) developed a web application for guided logic proof practice. They argue that, due to the lack of quick feedback from professors, an accessible ATP will improve students' understanding and produce better assessment results as it can provide specific feedback. Their paper also emphasized that, while a student may neglect to ask their professor questions out of fear of judgment, their ATP is non-judgmental and unbiased. However, while LogicLearner provides practice of logic proofs, our ATP parses an expression and checks for validity.

Aliyev's (2024) research on a programming language called LEAN 4 also shows the significance of ATPs. In addition to simple proposition proofs, ATPs can handle set theory and elementary algebra. LEAN 4 can aid understanding of set theory even for users with little coding experience. Unlike LogicLearner's developers, Aliyev (2024) did not directly create the system but instead shows how LEAN 4, developed by Leonardo de Moura, is useful for learning core topics in discrete mathematics. The main goal of his research is to provide an introduction for mathematicians who want to use computers to solve complex problems.

ATPs can also utilize Machine Learning (ML) to mimic the functionality of the human brain. This is incredibly useful for solving mathematical problems. Wei (2022) notes that research on ML methods in ATPs has increased in recent years. Specifically, research describes the use of deep neural networks. The neural network takes the proof goal and premises as input

and the proof tactic chosen at the current step as output. Wei's research shows that pure neural network models improve the overall success rate of ATPs.

Like Wei (2022), Holden (2021) highlights the importance of machine learning by summarizing research rather than developing a theorem prover. He discusses how ATPs can incorporate Machine Learning (ML) in order to solve SAT questions. The SAT or Boolean satisfiability problem asks how one might set the variables in a given Boolean formula so that it evaluates to TRUE. Holden's main goal is to highlight the current advances made on the topic at the time of his writing to help provide a building block for new work. Unlike Inamdar et al., Holden did not create an ATP but instead wishes to contribute to current work improving and advancing ATPs. He argues that many provers would benefit from the use of ML. Overall, the research on automated theorem provers demonstrates their value in the field of computer science and explains how enhancements can improve their effectiveness.

3 Methodology

3.1 System Overview

Our ATP, ValidityProver, is a propositional logic problem solver designed to handle basic logical operators and evaluate the validity of logical arguments. It utilizes a binary tree structure to parse and process logical expressions, generate truth tables, and assess logical validity. The key operators supported include *and* ($\wedge$), *or* ($\vee$), *implies* ($\rightarrow$), and *not* ($\neg$). The system evaluates statements, checks for tautologies, contradictions, and contingencies, and validates logical arguments through various reasoning rules, such as modus ponens, universal modus ponens, modus tollens, inverse error, and converse error.

3.2 Data Structures

The main data structure used in the program is a binary tree, implemented with the *TreeNode* class. Each *TreeNode* object contains:

- ➔ *data*: The logical operator or variable at the node.
- ➔ *left*: A reference to the left child node.
- ➔ *right*: A reference to the right child node.

The tree structure is designed to recursively parse and store the logical expressions with internal methods supporting traversal and leaf node extraction.

```
#Node class for binary tree
class TreeNode:
    #constructor
    def __init__(self, data, right=None, left=None):
        self.data = data
        self.left = left
        self.right = right
    #print tree for testing
    def print_tree(self, level=0):
       if self.data is not None:
            print("  " * level + str(self.data[0]))  # Indentation for levels
            if self.left:
                self.left.print_tree(level + 1)  # Print left subtree
            if self.right:
                self.right.print_tree(level + 1)  # Print right subtree


     #traverse through
    def getLeaves(self):
       if self.left is None and self.right is None:
          return [self.data[0]], [self]
       data_list = []
       leaf_list = []
       if self.left:
        left_data, left_nodes = self.left.getLeaves()
        data_list.extend(left_data)
        leaf_list.extend(left_nodes)

       if self.right:
        right_data, right_nodes = self.right.getLeaves()
        data_list.extend(right_data)
        leaf_list.extend(right_nodes)

       return data_list, leaf_list

    #returns true if node is a left leaf in the tree
    def isLeft(self, root):
        data, leafList = root.left.getLeaves()
        if self in leafList:
          return True
        else:
          return False
```

Figure 1. Tree Node class implementation in the ValidityProver

3.3 Expression Parsing

The *parse_expression* function converts natural language logical expressions into a formalized propositional logic format. It identifies logical operators (*and*, *or*, *implies*, *not*) and assigns corresponding symbols (∧, ∨, →, ¬). Additionally, it can handle universal and existential quantifiers, replacing the words "for all x" or "there exists an x such that" with ∀x or ∃x. The parser also manages expressions with variables, ensuring proper parentheses are in place to indicate logical precedence. It splits the expression into smaller components, assigns variables, and replaces the terms with appropriate symbols.

```
# Set of operators and variables - supports up to 4 parts
operators = ['→', '∨', '∧']
variables = ['A', 'B', 'C', 'D']

# Parser to tokenize and handle logical operators
def parse_expression(expression, vars):
#assume each sentence contains three parts
    #supports AND (∧), OR (∨), NOT (¬), IMPLIES (→) for now
    indices = [0]*10
    count = 1
    parentheses = [0]*4
    p=0
    labels = {'A': ' ', 'B': ' ', 'C': ' ', 'D': ' '}

    expr = expression.split()
    if expr[0] == "For" and expr[1] == "all" and expr[2] == "x":
      expr = expr[3:]
      expression = expression.replace("For all x", "∀x,")
    elif expr[0] == "There" and expr[1] == "exists" and expr[2] == "an" and expr[3] == "x" and expr[4] == "such" and expr[5] == "that":
      expr = expr[3:]
      expression = expression.replace("For all x", "∀x,")
    #loop through words of expression to convert to variables
    for i, w in enumerate(expr):
        if w == "implies" or w == "or" or w=="and" or i == len(expr)-1:
            indices[count] = i+1
            if i == len(expr)-1:
              ex = ' '.join(expr[indices[count-1]:indices[count]])
            else:
              ex = ' '.join(expr[indices[count-1]:indices[count]-1])
            labels[vars[count-1]] = ex
            if ex.__contains__('not'):
              expression = expression.replace(ex, '¬' + vars[count-1])
            else:
             expression = expression.replace(ex, vars[count-1])
            count = count+1

    exIndex1 = 0
    exIndex2 = 0
```

Figure 2.1. Expression parsing in the ValidityProver

```
#account for parentheses
for k, v in labels.items():
    if "(" in v:
        for e in expression:
          if e == k and expression[exIndex1-1] == '¬':
            expression = expression[:exIndex1-1] + "(" + expression[exIndex1-1:]
          elif e == k:
            expression = expression[:exIndex1] + "(" + expression[exIndex1:]
          exIndex1 = exIndex1+1
    if ")" in v:
        for e in expression:
          if e == k:
            expression = expression[:exIndex2+1] + ")" + expression[exIndex2+1:]
          exIndex2 = exIndex2+1

#replace operator words with symbols
expression = expression.replace('and', '∧').replace('or', '∨').replace('implies', '→')
return expression, labels
```

Figure 2.2. Expression parsing in the ValidityProver

3.4 Tree Construction

Once the expression is parsed, the *make_tree* function builds a binary tree representation of the logical expression. The function works recursively by identifying operators and creating tree nodes that represent both operands (as left and right nodes) and the logical operator. The function ensures that parentheses are handled correctly and subexpressions are processed before their operators.

```python
#creates a binary tree using a parsed expression
def make_tree(parsed_expression, labels):

    #if empty, return
    if not parsed_expression:
      return TreeNode(None)

    parsed_expression = parsed_expression.replace('∃x,', '').replace('∀x,', '')
    #if the whole expression is enclosed by parentheses, return only what's within
    if parsed_expression[0] == '(' and parsed_expression[-1] == ')' and len(parsed_expression)>1:
        return make_tree(parsed_expression[1:-1], labels)


    #if the expression is only a variable, create leaf node
    if len(parsed_expression) <= 2:
      for v in variables:
        if v in parsed_expression:
          if '¬' in parsed_expression:
            vList = ['¬' + v, labels[v]]
            return TreeNode(data=vList)
          else:
           vList = [v,labels[v] ]
           return TreeNode(data=vList)


    #loop which traverses the whole expression by each operator and recusrsively builds the tree
    for o in operators:
      depth = 0
      for p in range(len(parsed_expression) -1, -1, -1):
          c = parsed_expression[p]
          #test for parentheses/precedence
          if c == ')': depth +=1
          elif c == '(': depth -=1
          elif depth == 0 and c == o:
              left = make_tree(parsed_expression[:p-1], labels )
              right = make_tree(parsed_expression[p+2:], labels)
              node1 =TreeNode(c, right, left)
              return node1
```

Figure 3. Tree construction in ValidityProver

3.5 Truth Table Generation

The *generate_truth_table* function generates a truth table for the given logical expression. It accounts for all possible combinations of truth values for the variables involved and evaluates the expression for each combination. The *evaluate_expression* function replaces the variables

with their corresponding truth values, and the expression is evaluated using the Python *eval* function, which is extended to support logical operations.

```python
# Function to generate the truth table for a given expression
def generate_truth_table(expression, variables):
    # Create a list of all possible truth values for the variable
    truth_values = list(itertools.product([False, True], repeat=len(variables)))


    table = []
    for values in truth_values:
        assignment = dict(zip(variables, values))
        truth_value = evaluate_expression(expression, assignment)
        table.append((assignment, truth_value))


    return table


# Function to evaluate the expression given the truth values of the variables
def evaluate_expression(expression, assignment):
    # Replace the variables in the expression with their corresponding truth values
    for var, value in assignment.items():
        expression = expression.replace(var, str(value))


    # Evaluate the logical expression using Python's eval (with safety checks)
    try:
        return eval_expression(expression)
    except:
        return "Error in evaluation"


# Evaluate logical expressions
def eval_expression(expression):
    # Implement basic logical operations here
    expression = expression.replace('¬', 'not ')  # NOT
    expression = expression.replace('∧', ' and ')  # AND
    expression = expression.replace('∨', ' or ')   # OR
    expression = expression.replace('→', ' <= ')   # IMPLIES (A → B is equivalent to not A or B)
    return eval(expression)
```

Figure 4. Constructing a Truth Table

3.6 Quantified Statements and Modular Arithmetic Proofs (Extra Credit)

For the extra credit component, the system supports quantified statements and modular arithmetic proofs. The program handles quantified statements by checking for quantifiers and

adding them to the parsed statement appropriately. It checks for universal instantiation as well. Additionally, the program can evaluate modular expressions of the form:

- a ≡ b mod m and c ≡ d mod *m* implies (a+c) ≡ (b+d) mod *m*.

ValidityProver checks if the differences (a-b)%*m* and (c-d)%*m* are both 0, indicating that both a ≡ b mod *m* and c ≡ d mod *m*. If true, it computes whether (a+c) - (b+d) is divisible by *m* as well. The result is returned as either *true* or *false* based on whether the modular arithmetic condition holds. While the condition a ≡ b mod m and c ≡ d mod m should indicate that a+c ≡ b+d mod m is true, the function checks this to demonstrate the implication.

For instance, the proof checks that:

- 6 ≡ 2 mod 4 and 3 ≡ 7 mod 4, and then determines if 6 + 3 ≡ 2 + 7 mod 4

This feature allows for the validation of modular arithmetic theorems within the logical framework. If the given argument is valid, the program notes that it is valid not only by modus ponens but by universal modus ponens / instantiation.

```
#for extra credit modular arithmetic proofs
def modularProof(a, b, c, d, m):
  if (a-b)%m == 0 and (c-d)%m == 0:
    print(f"Yes, {a} - {b} is divisible by {m} and {c} - {d} is divisble by {m}")
    return ((a+c) - (b+d))%m == 0
  else:
    print(f"Either {a} - {b} is not divisible by {m} or {c} - {d} is not divisble by {m}, or both")
    return False
```

Figure 5. Extra credit: modular arithmetic proof

3.7 Argument Validity

The validity of an argument is determined using the *check_validity* function, which examines the truth table to assess whether the expression is a tautology, contradiction, or contingency. The *findArgumentValidity* function compares the truth values of the premises and conclusion, checking for specific reasoning errors such as modus ponens, modus tollens, inverse error, and converse error.

```
# Check if a statement is a tautology (always true), contradiction (always false), or contingency (sometimes true)
def check_validity(truth_table):
    all_true = all(truth for _, truth in truth_table)
    all_false = all(not truth for _, truth in truth_table)


    if all_true:
        return "Tautology (Always True)"
    elif all_false:
        return "Contradiction (Always False)"
    else:
        return "Contingency (Sometimes True)"
```

Figure 6.1. Checking whether the expression is a Tautology, Contradiction, or Contingency (part of provided code)

```python
#uses truth tables to determine validity
def findArgumentValidity(ex_table, prem_table, concl_table):
  matches = False
  valid = False
  for eAssignment, eTruthValue in ex_table:
    for pAssignment, pTruthValue in prem_table:
      for p in pAssignment:
          if pAssignment[p] == eAssignment[p]:
            matches = True
          else:
           matches = False
           break
      if matches == True:
        break
    for cAssignment, cTruthValue in concl_table:
      for c in cAssignment:
         if cAssignment[c] == eAssignment[c]:
            matches = True
         else:
           matches = False
           break
      if matches == True:
        break

    if matches == True and pTruthValue == True and eTruthValue == True and cTruthValue == False:
        return False
  return True
```

Figure 6.2. Checking Validity

```python
  # Check the validity of the expression
validity = check_validity(truth_table)
print(f"\nThe expression is a: {validity}")

option = False
print("Using the tables, we can deduce the validity of the entire argument - it is invalid only if both premises are \ntrue and the conclusion is false")
valid = findArgumentValidity(truth_table, prem_table, concl_table)
if valid == True:
  if prem_pe in negateExpression(right):
    print("Argument is valid by modus tollens")
    option = True
  elif prem_pe in left:
    if num2 ==1:
      print("Argument is valid by modus ponens")
      option = True
    elif num2==2:
       print("Argument is valid by universal modus ponens")
       option = True
elif valid == False:
  if prem_pe in negateExpression(left):
    print("Argument is invalid by inverse error")
    option = True
  elif prem_pe in right:
    print("Argument is invalid by converse error")
    option = True
if option == False:
  print("Argument is invalid")
```

Figure 6.3. Using the Truth Table to formulate a formal answer

3.8 Negation of Logical Statements

The system also supports the negation of logical expressions using the *negateExpression* function. This function converts each logical operator to its negated counterpart (e.g., *and* becomes *or*, and vice versa) and negates each variable using the ¬ symbol.

```
#negates a logical expression
def negateExpression(pExpr):
  pExpr = pExpr.replace("(", "( ").replace(")", " )")
  ex = pExpr.split()
  negated = ""
  for e in ex:
    if e in variables:
      if '¬' not in e:
        negated += '¬' + e + ' '
      else:
        negated += e.replace('¬', '')
    if e == '(':
      negated += '('
    if e == ')':
      negated += ')'
    if e == '→':
      negated += '∧ '
    if e == '∧':
      negated += '∨ '
    if e == '∨':
      negated += '∧ '
    if e == '∃x,':
      negated += '∀x '
    if e == '∀x,':
      negated += '∃x '
  return negated
```

Figure 7. Negating an expression

3.9 Main Function

The *main* function serves as the entry point for running the logic solver. It accepts an input logical expression (in natural language or symbolic form), parses it, builds the corresponding binary tree, generates the truth table, and validates logical arguments. It then

outputs the evaluation result, whether the logical expression is a tautology, contradiction, or contingency, and checks the validity of arguments provided.

3.10 User Experience

The user simply has to edit the given examples set in the original code. No additional coding is necessary making ValidityProver beginner friendly.

```
# Example: Propositional Logic Problem Solver
def main():

    print("This program is called ValidityProver and acts as a theorem parser and validity checker.\nIt evaluates different
    print("Each statement is written using the word 'implies' but the statement can also be expressed in if-then form.")
    # Modus ponens example - include paretheses to indicate priority
    print("\nExample 1:")
    modusPonensP1 = "Giving a mouse a cookie implies (he will ask for milk and get sick)"
    modusPonensP2 = "I gave the mouse a cookie"
    modusPonensConcl = "He will ask for milk and get sick"
    runProgram(modusPonensP1, modusPonensP2, modusPonensConcl, 1, 1)

    #modus tollens example
    print("\nExample 2:")
    modusTollensP1 = "(Sarah laying in the sun and getting tan) implies she is happy"
    modusTollensP2 = "She is not happy"
    modusTollensConcl = "Sarah does not lay in the sun or does not get tan"
    runProgram(modusTollensP1, modusTollensP2, modusTollensConcl, 2, 1)

    #inverse error example
    print("\nExample 3:")
    converseP1 = "Riding the train implies (you have a car or you do not have a bike)"
    converseP2 = "You have a car"
    converseConcl = "You ride the train"
    runProgram(converseP1, converseP2, converseConcl, 2, 1)


    #converse error example
    print("\nExample 4:")
    inverseP1 = "The student being in the library implies (they are not doing well in class and need a tutor)"
    inverseP2 = "The student is not in the library"
    inverseConcl = "They are doing well in class and do not need a tutor"
    runProgram(inverseP1, inverseP2, inverseConcl, 1, 1)


    #Quantified statement example
    print("\nExample 5:")
    quantifyP1 = "For all x, x being > 0 implies that it is a natural number"
    quantifyP2 = "7 is > 0"
    quantifyConcl = "7 is a natural number"
    runProgram(quantifyP1, quantifyP2, quantifyConcl, 1, 2)
```

Figure 8.1. Different examples to show the types of expressions ValidityProver can handle

```python
#Modular arithmetic proofs example
print("\nExample 6:")
modularP1 = "a ≡ b mod m and c ≡ d mod m implies a + c ≡ b + d mod m"
modularP2 = "6 ≡ 2 mod 4 and 3 ≡ 7 mod 4"
modularConcl = "4 + 3 ≡ 2 + 7 (mod 4)"
print("The statement below means that if (a-b) % m is equal to zero and if (c-d) % m is equal to 0, then ((a+c)-(b+d)) % m equals 0")
print("That is, they are divisible by m")
runProgram(modularP1, modularP2, modularConcl, 1, 2)
print("\nBut we must check if it is true for the given values: ")
isValid = modularProof(6, 2, 3, 7, 4)


#negation demonstration
print("\nExample 7: ")
print("Demonstration of negation of conditional quantified statement: ")
statement = "There exists an x such that multiplying by 10 implies (x will equal 120 and be divisible by 3)"
parsed_statement, sLabels = parse_expression(statement, variables)
print("Statement: " + statement.replace("(", "").replace(")", ""))
print("Parsed, this statement is " + parsed_statement)
negated_statement = negateExpression(parsed_statement)
print("Negated, the statement becomes " +negated_statement)

#tautology example
print("\nExample 8: ")
print("Demonstration of a tautology:")
tautology = "I go to the store or I do not go to the store"
parsed_tautology, tLabels = parse_expression(tautology, ['A', 'A'])
print("Statement: " + tautology.replace("(", "").replace(")", ""))
print("Parsed, this statement is " + parsed_tautology)
tautology_table = generate_truth_table(parsed_tautology, ['A', 'A'])
print("Truth table:")
for assignment, truth_value in tautology_table:
    print(f"Variables: {assignment} | Truth Value: {truth_value}")
print("This statement is a " + check_validity(tautology_table))

#contradiction example
print("\nExample 9: ")
print("Demonstration of a contradiction:")
contradiction = "Tonight I ate dinner and I did not eat dinner"
parsed_contradiction, cLabels = parse_expression(contradiction, ['A', 'A'])
print("Statement: " + contradiction.replace("(", "").replace(")", ""))
print("Parsed, this statement is " + parsed_contradiction)
contradiction_table = generate_truth_table(parsed_contradiction, ['A', 'A'])
print("Truth table:")
for assignment, truth_value in contradiction_table:
    print(f"Variables: {assignment} | Truth Value: {truth_value}")
print("This statement is a " + check_validity(contradiction_table))
```

Figure 8.2. Different examples to show the types of expressions ValidityProver can handle

4 Results

```
This program is called ValidityProver and acts as a theorem parser and validity checker.
It evaluates different types of arguments and assesses both their validity and
the type of argument or error that might be present. It does this by logically parsing
the expressions, creating a binary tree to represent them, and providing a truth table for each one.
Each statement is written using the word 'implies' but the statement can also be expressed in if-then form.

Example 1:

Conditional statement/premise 1: Giving a mouse a cookie implies he will ask for milk and get sick
Premise 2: I gave the mouse a cookie
Conclusion 2: He will ask for milk and get sick

We parse the expression and create a tree and truth tables:
The first premise parsed is A → (B ∧ C)
The tree for the expression is:
→
  A
  ∧
    B
    C

Truth Table for Premise 1:
Expression: A → (B ∧ C) | Variables: {'A': False, 'B': False, 'C': False} |  | (B ∧ C): False| Truth Value: True
Expression: A → (B ∧ C) | Variables: {'A': False, 'B': False, 'C': True} |  | (B ∧ C): False| Truth Value: True
Expression: A → (B ∧ C) | Variables: {'A': False, 'B': True, 'C': False} |  | (B ∧ C): False| Truth Value: True
Expression: A → (B ∧ C) | Variables: {'A': False, 'B': True, 'C': True} |  | (B ∧ C): True| Truth Value: True
Expression: A → (B ∧ C) | Variables: {'A': True, 'B': False, 'C': False} |  | (B ∧ C): False| Truth Value: False
Expression: A → (B ∧ C) | Variables: {'A': True, 'B': False, 'C': True} |  | (B ∧ C): False| Truth Value: False
Expression: A → (B ∧ C) | Variables: {'A': True, 'B': True, 'C': False} |  | (B ∧ C): False| Truth Value: False
Expression: A → (B ∧ C) | Variables: {'A': True, 'B': True, 'C': True} |  | (B ∧ C): True| Truth Value: True

Truth Table for Premise 2:
Expression: A | Variables: {'A': False} | Truth Value: False
Expression: A | Variables: {'A': True} | Truth Value: True

Truth Table for Conclusion:
Expression:  B ∧ C | Variables: {'B': False, 'C': False} | Truth Value: False
Expression:  B ∧ C | Variables: {'B': False, 'C': True} | Truth Value: False
Expression:  B ∧ C | Variables: {'B': True, 'C': False} | Truth Value: False
Expression:  B ∧ C | Variables: {'B': True, 'C': True} | Truth Value: True

The expression is a: Contingency (Sometimes True)
Using the tables, we can deduce the validity of the entire argument - it is invalid only if both premises are
true and the conclusion is false
Argument is valid by modus ponens
```

Figure 9. Results for example #1

```
Example 2:

Conditional statement/premise 1: Sarah laying in the sun or getting tan implies she is happy
Premise 2: She is not happy
Conclusion 2: Sarah does not lay in the sun and does not get tan

We parse the expression and create a tree and truth tables:
The first premise parsed is (A ∨ B) → C
The tree for the expression is:
→
  ∨
    A
    B
  C

Truth Table for Premise 1:
Expression: (A ∨ B) → C | Variables: {'A': False, 'B': False, 'C': False} |(A ∨ B) : False|  | Truth Value: True
Expression: (A ∨ B) → C | Variables: {'A': False, 'B': False, 'C': True} |(A ∨ B) : False|  | Truth Value: True
Expression: (A ∨ B) → C | Variables: {'A': False, 'B': True, 'C': False} |(A ∨ B) : True|  | Truth Value: False
Expression: (A ∨ B) → C | Variables: {'A': False, 'B': True, 'C': True} |(A ∨ B) : True|  | Truth Value: True
Expression: (A ∨ B) → C | Variables: {'A': True, 'B': False, 'C': False} |(A ∨ B) : True|  | Truth Value: False
Expression: (A ∨ B) → C | Variables: {'A': True, 'B': False, 'C': True} |(A ∨ B) : True|  | Truth Value: True
Expression: (A ∨ B) → C | Variables: {'A': True, 'B': True, 'C': False} |(A ∨ B) : True|  | Truth Value: False
Expression: (A ∨ B) → C | Variables: {'A': True, 'B': True, 'C': True} |(A ∨ B) : True|  | Truth Value: True

Truth Table for Premise 2:
Expression: ¬C | Variables: {'C': False} | Truth Value: True
Expression: ¬C | Variables: {'C': True} | Truth Value: False

Truth Table for Conclusion:
Expression:  ¬A ∧ ¬B | Variables: {'A': False, 'B': False} | Truth Value: True
Expression:  ¬A ∧ ¬B | Variables: {'A': False, 'B': True} | Truth Value: False
Expression:  ¬A ∧ ¬B | Variables: {'A': True, 'B': False} | Truth Value: False
Expression:  ¬A ∧ ¬B | Variables: {'A': True, 'B': True} | Truth Value: False

The expression is a: Contingency (Sometimes True)
Using the tables, we can deduce the validity of the entire argument - it is invalid only if both premises are true and the conclusion is false
Argument is valid by modus tollens
```

Figure 10. Results for example #2

```
Example 3:

Conditional statement/premise 1: Riding the train implies you have a car or you do not have a bike
Premise 2: You do not have a bike
Conclusion 2: You ride the train

We parse the expression and create a tree and truth tables:
The first premise parsed is A → (B ∨ ¬C)
The tree for the expression is:
→
  A
  ∨
    B
    C

Truth Table for Premise 1:
Expression: A → (B ∨ ¬C) | Variables: {'A': False, 'B': False, 'C': False} |  | (B ∨ ¬C): True| Truth Value: True
Expression: A → (B ∨ ¬C) | Variables: {'A': False, 'B': False, 'C': True} |  | (B ∨ ¬C): False| Truth Value: True
Expression: A → (B ∨ ¬C) | Variables: {'A': False, 'B': True, 'C': False} |  | (B ∨ ¬C): True| Truth Value: True
Expression: A → (B ∨ ¬C) | Variables: {'A': False, 'B': True, 'C': True} |  | (B ∨ ¬C): True| Truth Value: True
Expression: A → (B ∨ ¬C) | Variables: {'A': True, 'B': False, 'C': False} |  | (B ∨ ¬C): True| Truth Value: True
Expression: A → (B ∨ ¬C) | Variables: {'A': True, 'B': False, 'C': True} |  | (B ∨ ¬C): False| Truth Value: False
Expression: A → (B ∨ ¬C) | Variables: {'A': True, 'B': True, 'C': False} |  | (B ∨ ¬C): True| Truth Value: True
Expression: A → (B ∨ ¬C) | Variables: {'A': True, 'B': True, 'C': True} |  | (B ∨ ¬C): True| Truth Value: True

Truth Table for Premise 2:
Expression: ¬B | Variables: {'B': False} | Truth Value: True
Expression: ¬B | Variables: {'B': True} | Truth Value: False

Truth Table for Conclusion:
Expression:  A | Variables: {'A': False} | Truth Value: False
Expression:  A | Variables: {'A': True} | Truth Value: True

The expression is a: Contingency (Sometimes True)
Using the tables, we can deduce the validity of the entire argument - it is invalid only if both premises are true and the conclusion is false
Argument is invalid
```

Figure 11. Results for example #3

```
Example 4:

Conditional statement/premise 1: The student being in the library implies they are not doing well in class and need a tutor
Premise 2: The student is not in the library
Conclusion 2: They do not need a tutor

We parse the expression and create a tree and truth tables:
The first premise parsed is A → (¬B ∧ C)
The tree for the expression is:
→
  A
  ∧
    B
    C

Truth Table for Premise 1:
Expression: A → (¬B ∧ C) | Variables: {'A': False, 'B': False, 'C': False} |  | (¬B ∧ C): False| Truth Value: True
Expression: A → (¬B ∧ C) | Variables: {'A': False, 'B': False, 'C': True} |  | (¬B ∧ C): True| Truth Value: True
Expression: A → (¬B ∧ C) | Variables: {'A': False, 'B': True, 'C': False} |  | (¬B ∧ C): False| Truth Value: True
Expression: A → (¬B ∧ C) | Variables: {'A': False, 'B': True, 'C': True} |  | (¬B ∧ C): False| Truth Value: True
Expression: A → (¬B ∧ C) | Variables: {'A': True, 'B': False, 'C': False} |  | (¬B ∧ C): False| Truth Value: False
Expression: A → (¬B ∧ C) | Variables: {'A': True, 'B': False, 'C': True} |  | (¬B ∧ C): True| Truth Value: True
Expression: A → (¬B ∧ C) | Variables: {'A': True, 'B': True, 'C': False} |  | (¬B ∧ C): False| Truth Value: False
Expression: A → (¬B ∧ C) | Variables: {'A': True, 'B': True, 'C': True} |  | (¬B ∧ C): False| Truth Value: False

Truth Table for Premise 2:
Expression: ¬A | Variables: {'A': False} | Truth Value: True
Expression: ¬A | Variables: {'A': True} | Truth Value: False

Truth Table for Conclusion:
Expression:  ¬B | Variables: {'B': False} | Truth Value: True
Expression:  ¬B | Variables: {'B': True} | Truth Value: False

The expression is a: Contingency (Sometimes True)
Using the tables, we can deduce the validity of the entire argument - it is invalid only if both premises are true and the conclusion is false
Argument is invalid by inverse error
```

Figure 12. Results for example #4

```
Example 5:

Conditional statement/premise 1: For all x, x being > 0 implies that it is a natural number
Premise 2: 7 is > 0
Conclusion 2: 7 is a natural number

We parse the expression and create a tree and truth tables:
The first premise parsed is ∀x, A → B
The tree for the expression is:
→
  A
  B

Truth Table for Premise 1:
Expression: ∀x, A → B | Variables: {'A': False, 'B': False} |∀x, A : False|  | Truth Value: True
Expression: ∀x, A → B | Variables: {'A': False, 'B': True} |∀x, A : False|  | Truth Value: True
Expression: ∀x, A → B | Variables: {'A': True, 'B': False} |∀x, A : False|  | Truth Value: False
Expression: ∀x, A → B | Variables: {'A': True, 'B': True} |∀x, A : False|  | Truth Value: True

Truth Table for Premise 2:
Expression: A | Variables: {'A': False} | Truth Value: False
Expression: A | Variables: {'A': True} | Truth Value: True

Truth Table for Conclusion:
Expression:  B | Variables: {'B': False} | Truth Value: False
Expression:  B | Variables: {'B': True} | Truth Value: True

The expression is a: Contingency (Sometimes True)
Using the tables, we can deduce the validity of the entire argument - it is invalid only if both premises are
true and the conclusion is false
Argument is valid by universal modus ponens / instantiation
```

Figure 13. Results for example #5

```
Example 6:
The statement below means that if (a-b) % m is equal to zero and if (c-d) % m is equal to 0, then ((a+c)-(b+d)) % m equals 0
That is, they are divisible by m

Conditional statement/premise 1: a ≡ b mod m and c ≡ d mod m implies a + c ≡ b + d mod m
Premise 2: 6 ≡ 2 mod 4 and 3 ≡ 7 mod 4
Conclusion 2: 4 + 3 ≡ 2 + 7 (mod 4)

We parse the expression and create a tree and truth tables:
The first premise parsed is A ∧ B → C
The tree for the expression is:
→
  ∧
    A
    B
  C

Truth Table for Premise 1:
Expression: A ∧ B → C | Variables: {'A': False, 'B': False, 'C': False} |A ∧ B : False|  | Truth Value: False
Expression: A ∧ B → C | Variables: {'A': False, 'B': False, 'C': True} |A ∧ B : False|  | Truth Value: False
Expression: A ∧ B → C | Variables: {'A': False, 'B': True, 'C': False} |A ∧ B : False|  | Truth Value: False
Expression: A ∧ B → C | Variables: {'A': False, 'B': True, 'C': True} |A ∧ B : False|  | Truth Value: False
Expression: A ∧ B → C | Variables: {'A': True, 'B': False, 'C': False} |A ∧ B : False|  | Truth Value: True
Expression: A ∧ B → C | Variables: {'A': True, 'B': False, 'C': True} |A ∧ B : False|  | Truth Value: True
Expression: A ∧ B → C | Variables: {'A': True, 'B': True, 'C': False} |A ∧ B : False|  | Truth Value: False
Expression: A ∧ B → C | Variables: {'A': True, 'B': True, 'C': True} |A ∧ B : False|  | Truth Value: True

Truth Table for Premise 2:
Expression: A ∧ B | Variables: {'A': False, 'B': False} | Truth Value: False
Expression: A ∧ B | Variables: {'A': False, 'B': True} | Truth Value: False
Expression: A ∧ B | Variables: {'A': True, 'B': False} | Truth Value: False
Expression: A ∧ B | Variables: {'A': True, 'B': True} | Truth Value: True

Truth Table for Conclusion:
Expression:  (C) | Variables: {'C': False} | Truth Value: False
Expression:  (C) | Variables: {'C': True} | Truth Value: True

The expression is a: Contingency (Sometimes True)
Using the tables, we can deduce the validity of the entire argument - it is invalid only if both premises are
true and the conclusion is false
Argument is valid by universal modus ponens / instantiation

But we must check if it is true for the given values:
Yes, 6 - 2 is divisible by 4 and 3 - 7 is divisble by 4
```

Figure 14. Results for example #6

```
Example 7:
Demonstration of negation of conditional quantified statement:
Statement: There exists an x such that multiplying by 10 implies x will equal 120 and be divisible by 3
Parsed, this statement is ∃x, A → (B ∧ C)
Negated, the statement becomes ∀x, ¬A ∧ (¬B ∨ ¬C )
```

Figure 15. Results for example #7

```
Example 8:
Demonstration of a tautology:
Statement: I go to the store or I do not go to the store
Parsed, this statement is A ∨ ¬A
Truth table:
Variables: {'A': False} | Truth Value: True
Variables: {'A': True} | Truth Value: True
Variables: {'A': False} | Truth Value: True
Variables: {'A': True} | Truth Value: True
This statement is a Tautology (Always True)
```

Figure 16. Results for example #8

```
Example 9:
Demonstration of a contradiction:
Statement: Tonight I ate dinner and I did not eat dinner
Parsed, this statement is A ∧ ¬A
Truth table:
Variables: {'A': False} | Truth Value: False
Variables: {'A': True} | Truth Value: False
Variables: {'A': False} | Truth Value: False
Variables: {'A': True} | Truth Value: False
This statement is a Contradiction (Always False)
```

Figure 17. Results for example #9

5 Discussion

ValidityProver successfully interprets logical arguments by translating natural language or symbolic input into a formal logic expression, constructing binary expression trees, and generating comprehensive truth tables. These truth tables are then used to determine whether the expression is a tautology, contradiction, or contingency and to assess the validity of logical arguments using forms such as modus ponens or modus tollens. The solver correctly identifies valid and invalid arguments by analyzing truth value consistency between premises and conclusions.

Across all test cases, which includes examples of valid and invalid deductive reasoning, ValidityProver was able to correctly parse and transform natural language expressions into symbolic logic, generate accurate truth tables that reflect all possible truth value combinations, evaluate and classify expressions based on their logical form, and detect common argument forms and identify when logical fallacies (for example: converse or inverse errors) occurred. Thus, ValidityProver can serve as an effective educational tool or foundational framework for more advanced logic analysis systems.

However, this prover does have limitations. First, the format of the input expression is very rigid, as the language parser has a fixed structure and can fail with more complex or irregular phrasing. Secondly, there is a limit on the type of logical operators the user can use. While supporting conjunction ($\wedge$), disjunction ($\vee$), negation ($\neg$), and implication ($\rightarrow$), the system does not yet handle biconditional logic ($\leftrightarrow$) or exclusive ($\oplus$). Additionally, if there is an error in a truth table evaluation, the program does not provide detailed user feedback. This may be challenging for beginners trying to understand why an expression does not evaluate to the wanted output. Lastly, the program cannot handle expressions that use more than 4 variables.

Conclusion

Automated theorem provers like ValidityProver can be a useful tool for not only future programmers but for those learning propositional logic. Creating easy-to-understand code is essential for students who may have never handled code before. While ValidityProver represents a simple design model, research on this area of computer science demonstrates how higher levels of computing, like machine learning, enable simple provers to handle more complex problems.

In conclusion, ValidityProver serves as a foundation to more advanced provers while still providing a valuable tool for those learning propositional logic.

References

Aliyev, Y. N. (2024). Set theory and elementary algebra in Lean 4 theorem prover. *2024 IEEE 18th International Conference on Application of Information and Communication Technologies (AICT)*, 1–6. https://doi.org/10.1109/aict61888.2024.10740429

Holden, S. B. (2021). *Machine Learning for Automated Theorem Proving: Learning to Solve Sat and QSAT*. https://doi.org/10.1561/9781680838992

Inamdar, A. et alt. (2025). LogicLearner: A Tool for the Guided Practice of Propositional Logic Proofs. https://arxiv.org/abs/2503.19280

Wei, Y. (2022). Applying autoencoder to automated theorem proving. *2022 4th International Conference on Frontiers Technology of Information and Computer (ICFTIC)*, 759–762. https://doi.org/10.1109/icftic57696.2022.10075262